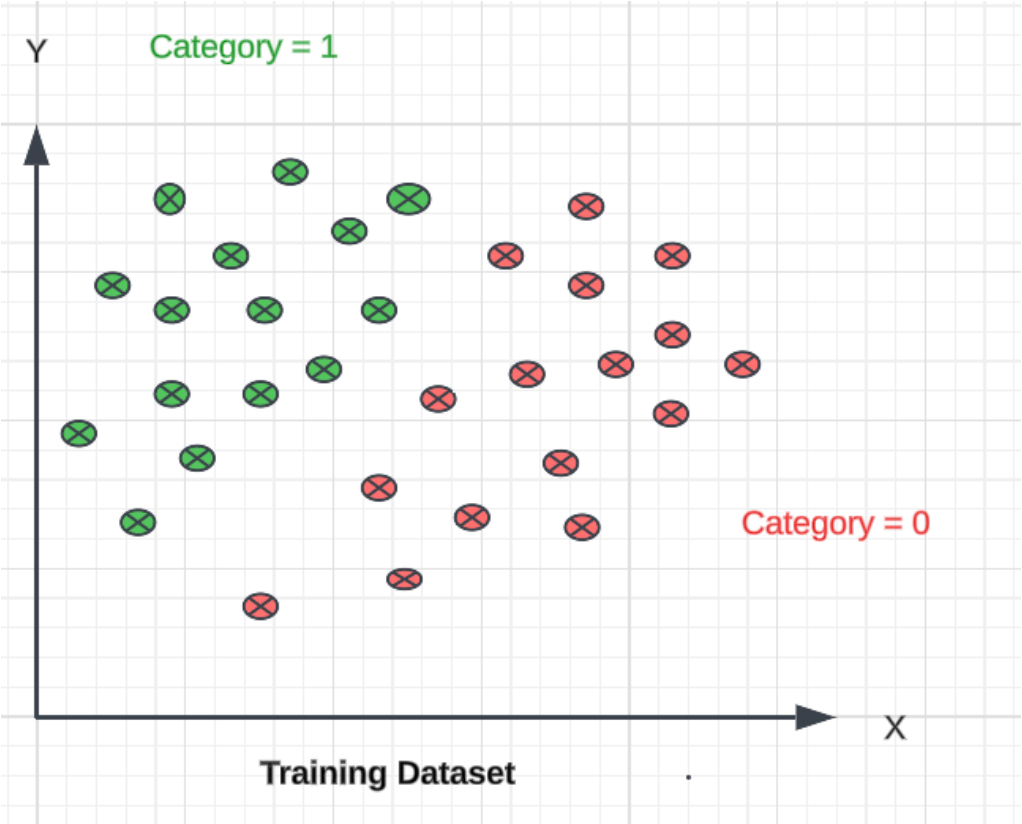
Non parametric Model

A **non-parametric algorithm** doesn't assume a **fixed shape for the data**, like a **line or curve.** Instead, **it adapts based on the actual data points available**.

This flexibility helps it work well even if the data has **complex patterns.**

K-Nearest Neighbour (KNN) - Classification

K-Nearest Neighbors (K-NN) is a simple, **non-parametric algorithm** used for **classification** and **regression.**



….

K-Nearest Neighbour (KNN)

Step1:

We have to initialize the K value

K can be **greater than 0** and to **any Number**

K>0…….

K=1,2,3,4,5,……..

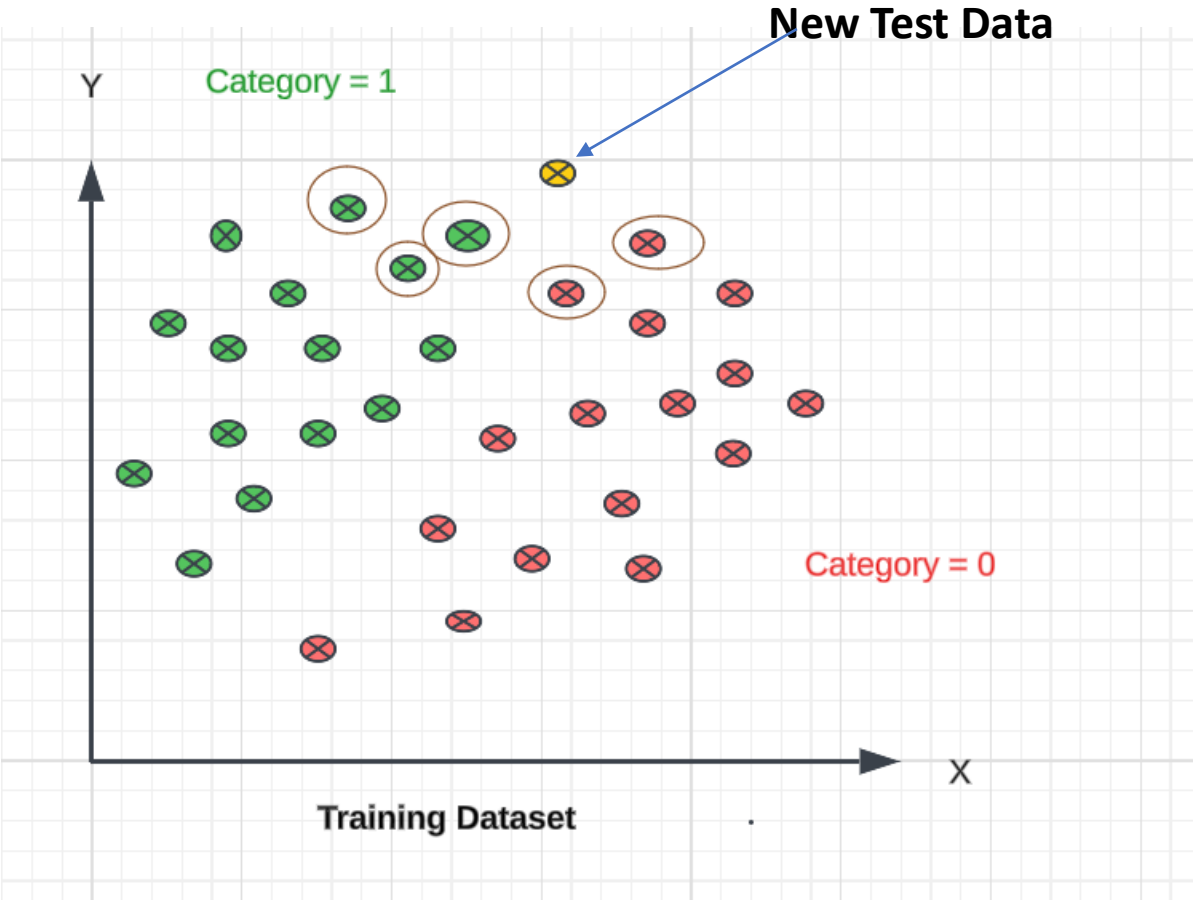
K is a **hyper parameter,** so that we can change the **K value** to different data set.

We have to **try** by changing **different K value** and we have to consider that particular **K value** which gives **highest accuracy .**

.....

Let us say we have chosen **K=5**

Step 2: Find the K Nearest neighbour for the New Test data



……

Step 3:

From the **K=5** values ,How many **neighbor** belong to **0 category** and how many belongs to **1 category**.

Category 1 = **3**

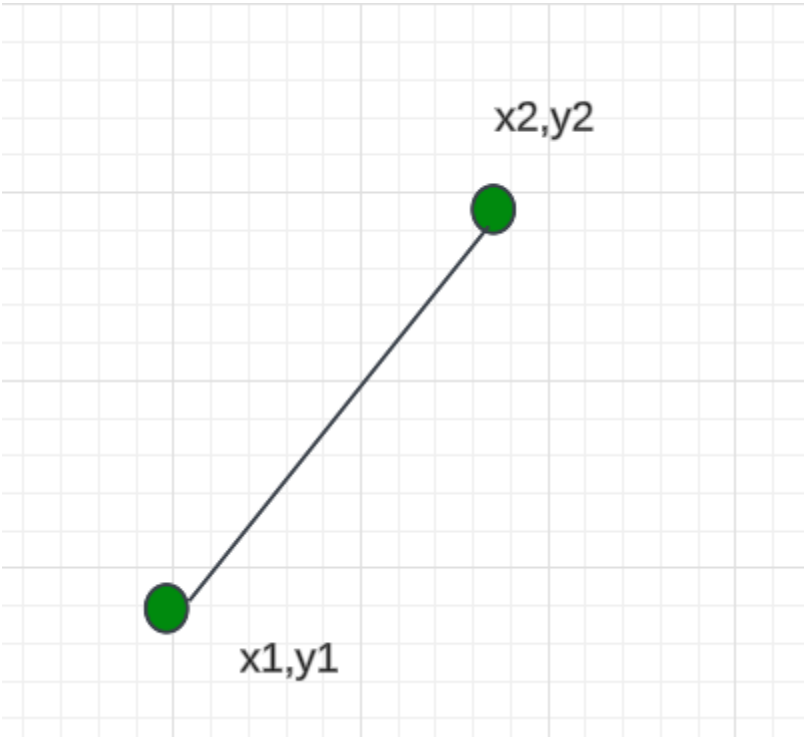
Category 0 = **2**

Since **Category 1** has more **neighbours** , we can **predict** that the **new data point** belongs to **category1**

……

To calculate the K-Nearest neighbour we will use **distance formula.**

1.Euclidian distance



Distance $=\sqrt{(x2-x1)^2+(y2-y1)^2}$

Facial recognition

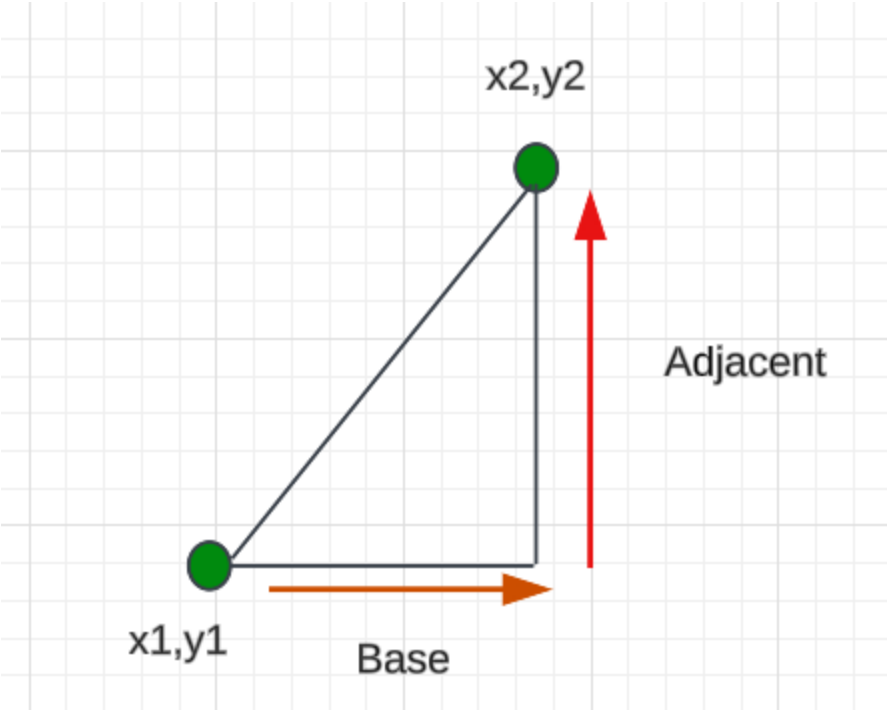
Two images are compared by calculating the **Euclidean distance** between their **feature vectors**.

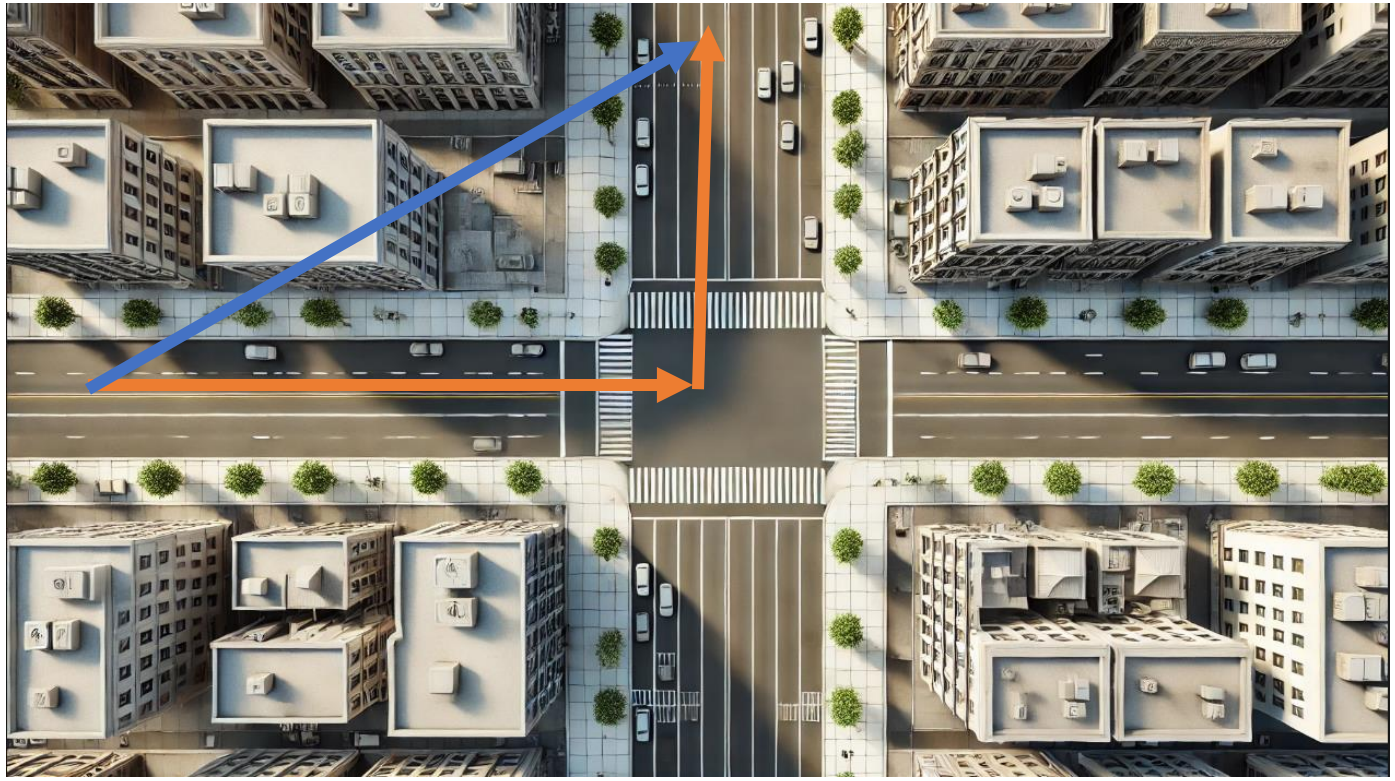
.....

2.Manhattan Distance

Also known "taxicab" distance

Distance = $|x2 - x1| + |y2 - y1|$





City Block Distance

Manhattan distance is ideal since the vehicles follow a **grid pattern**, **and diagonal shortcuts are not allowed.**

K-Nearest Neighbour (KNN) - Regression

If the K value =5

Average of all these points to find out the O/P of the New data point

k-Nearest Neighbour-Classifier

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[2]

Suggested code may be subject to a license | tayyabmalik4/Data_preprocessing_And_Feature_Engineering_With_Tayyab

```
#importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#import datsets
from sklearn.datasets import make_classification
x,y=make_classification(n_samples=1000,n_features=5,n_classes=2,random_state=42)
```

k-Nearest Neighbour-Classifier

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[9]

```python
#Train test split
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=42)
```

```python
#Import K-Nearest Neignbour classifier

from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=5,algorithm='auto')
knn.fit(x_train,y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

Model will consider **the 5 nearest neighbors** of any new data point to predict its class label.

Allows the model to select the **most efficient algorithm** automatically (either **ball tree, kd_tree, or brute force**), based on the **data size and structure**.

k-Nearest Neighbour-Classifier

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[11]
```python
#prediction
y_pred=knn.predict(x_test)
```

Suggested code may be subject to a license | BHATNISAR/MARKETING-LEAD-CLASSIFIER

```python
#perfomance metrics
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report
print(accuracy_score(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))
```

```
0.8966666666666666
[[137   9]
 [ 22 132]]
              precision    recall  f1-score   support

           0       0.86      0.94      0.90       146
           1       0.94      0.86      0.89       154

    accuracy                           0.90       300
   macro avg       0.90      0.90      0.90       300
weighted avg       0.90      0.90      0.90       300
```

Use **k-d trees** for low-dimensional(<20), large datasets.
Use **Ball tree** for Moderate (20-30),Large dataset
Use **brute-force search** for high-dimensional (>30) small to moderate datasets.

How to find the best K value?

Suggested code may be subject to a license | | Genedy-yonis/Step-by-Step-Diabetes-Classification-KNN-detailed

```python
#grid search
from sklearn.model_selection import GridSearchCV
# Define the parameter grid with possible values for n_neighbors
param_grid = {'n_neighbors': range(1, 31)}  # Testing values of K from 1 to 30

# Initialize the KNeighborsClassifier
knn = KNeighborsClassifier()

# Set up GridSearchCV without cross-validation (cv=1)
knn_cv = GridSearchCV(knn, param_grid, cv=5)

# Fit the model on the data
knn_cv.fit(x, y)

# Get the best K value
best_k = knn_cv.best_params_['n_neighbors']
print("Best K value:", best_k)
```

You define a **range** of possible values for each **hyperparameter** (like **n_neighbors** in **KNN)** in a **parameter grid**.

```
Best K value: 10
/usr/local/lib/python3.10/dist-packages/numpy/ma/core.py:2820: RuntimeWarning: invalid value encountered in cast
  _data = np.array(data, dtype=dtype, copy=copy,
```

Bayes Theorem

Probability: Is a measure of how likely an event is to happen

Take a Coin

S ={Head, Tail}

n(S) => Total No of Chances = 2

A = Event of Getting Head

n(A) => Chance of getting head = 1

$$P(A) = \frac{No\ of\ Chances\ for\ an\ Event}{Total\ No\ of\ Chances}$$

$$P(A) = \frac{1}{2} = 0.5$$

.....

Independent Events: The probability of one event occurring doesn't change based on the occurrence of the other.

For independent events A and B

Let A be the event of flipping a coin and getting heads.

Let B be the event of rolling a die and getting a 6.

These events are independent because the outcome of the coin flip doesn't influence the outcome of the die roll.

$$\mathbf{P(A \cap B) = P(A) \times P(B)}$$

This means the probability of both events happening together is simply the product of their individual probabilities.

……

Dependent Events: Where the occurrence of one event affects the probability of the other event happening.

For dependent events A and B, the probability of both occurring is:

$$P(A \cap B) = P(A) \times P(B|A)$$

P(B|A): The conditional probability of B given that A has already occurred.

P(A): Probability that event **A** occurs

P(A ∩ B): Probability that both **A** and **B** occur together.

.....

Conditional Probability: Is the probability of an event happening, given that another event has already occurred.

Event A: It's raining

Event B: You carry an umbrella.

Probability of **Event B** occurring, given that **Event A** has already occurred.

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

P(B|A): Conditional probability of B given A.

P(B ∩ A): Probability that both A and B occur together.

P(A): Probability that event **A** occurs.

.....

Bayes Theorem Derivation:

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

Conditional probability of **B** given **A**.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Conditional probability of **A** given **B.**

$$P(A \cap B) = P(B \cap A)$$

Both are same

$$P(A|B) * P(B) = P(B|A) * P(A)$$

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Bayes Theorem Formula:

Likely hood

$$\mathbf{P(A|B)} = \frac{P(B|A) * P(A)}{P(B)}$$

Prior Probability

Posterior Probability

Marginal Probability

Naïve Bayes Classifier

The **Naive Bayes classifier** is a simple, **probabilistic classification algorithm** based on **Bayes' theorem**.

Types of Naive Bayes

- **Gaussian Naive Bayes**
- **Multinomial Naive Bayes**
- **Bernoulli Naive Bayes**

……

Given Data set :

X = {x1 , x2 ,x3…….xn} **y**

Independent Features **Dependent Variable**

The formula for calculating the **probability** of a **class Y**

$$\mathbf{P(A|B)} = \frac{P(B|A) * P(A)}{P(B)}$$

$$\mathrm{P(Y/(x1,x2,x3\ldots..xn)} = \frac{P(x1|y)P(x2|y)P(x3|y)\ldots\ldots P(xn|y) * P(Y)}{P(x1,x2,x3\ldots..xn)}$$

Example- Naïve Bayes Classifier

Here's a sample dataset with features like **Outlook, Temperature, Humidity**, and **Windy** that are used to predict whether **Play Tennis** will be **Yes or No.**

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	[illegible]	True	No

.....

We want to **predict** Play Tennis for the following **conditions?**

Conditions=(Outlook = Sunny,Temperature = Cool,Humidity = High,Windy = True)

……

- **Step-by-Step Naive Bayes Calculation**

Step 1: Calculate **Prior Probabilities**

calculate the probability of each class P**(Play Tennis=Yes)** and P**(Play Tennis=No)**

Total Yes: 9 occurrences $P(\text{Yes}) = \frac{9}{14} = 0.64$

Total No: 5 occurrences $P(\text{No}) = \frac{5}{14} = 0.35$

Total records: 14

.....

Step 2: Calculate Likelihoods

Calculate the **likelihood** of each feature.

For Play Tennis = Yes:

$$P(\text{Outlook=Sunny}|\text{Yes}) = \frac{2}{9}$$

$$P(\text{Temperature=Cool}|\text{Yes}) = \frac{3}{9}$$

$$P(\text{Humidity=High}|\text{Yes}) = \frac{3}{9}$$

$$P(\text{Windy=True}|\text{Yes}) = \frac{3}{9}$$

……

For Play Tennis = No:

$P(\text{Outlook=Sunny} \mid \text{No})=\frac{3}{5}$

$P(\text{Temperature=Cool} \mid \text{No})= \frac{1}{5}$

$P(\text{Humidity=High} \mid \text{No})= \frac{4}{5}$

$P(\text{Windy=True} \mid \text{No})=\frac{2}{5}$

……

Step 3: Apply Bayes' Theorem

To classify, we find ***P*(Yes|conditions)** and ***P*(No|conditions)** and choose the one with the **higher probability**.

Conditions=(Outlook = Sunny,Temperature = Cool,Humidity = High,Windy = True)

$$\mathbf{P(Yes|conditions)} = \frac{\mathbf{P(Sunny|yes)P(cool|yes)P(High|yes)P(True|yes)*P(yes)}}{P(conditions)}$$

$$= \frac{2}{9}\ \frac{3}{9}\ \frac{3}{9}\ \frac{3}{9} * 0.64$$

$$= \mathbf{0.0053}$$

.....

P(No|Conditions) $= \frac{P(Sunny|No)P(cool|No)P(High|No)P(True|No)*P(No)}{P(conditions)}$

$= \frac{3}{5}\ \frac{1}{5}\ \frac{4}{5}\ \frac{2}{5} * 0.36$

$= \mathbf{0.0115}$

Step 4: Compare and Classify

Since ***P*(No|conditions)** > ***P*(Yes|conditions),** the classifier predicts **Play Tennis = No** for these conditions.

Naive Bayes uses **conditional probabilities** to **classify data.**

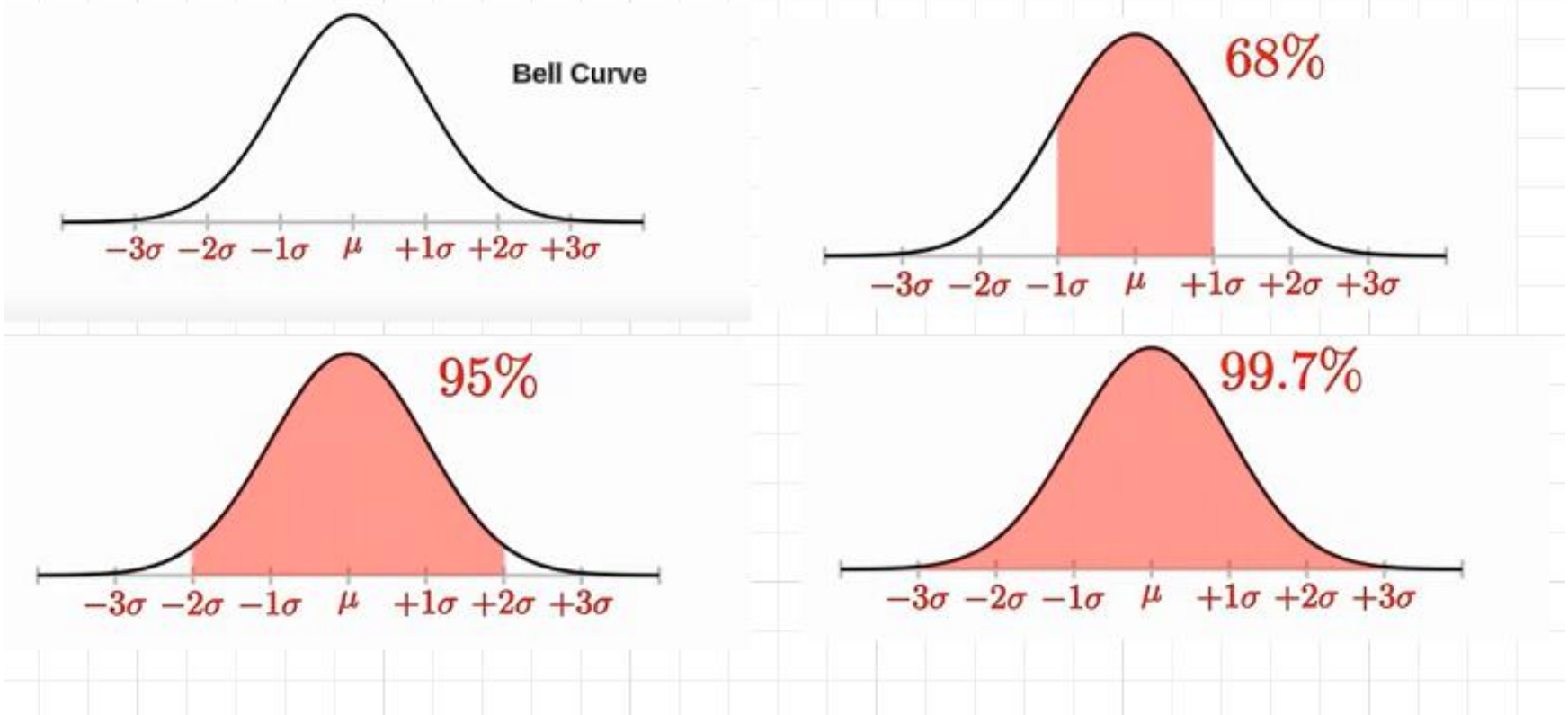
Marks

$\mu = 70$

$\sigma = 10$

Normal or Gaussian Distribution

In a normal distribution, the **mean (average)**,located at the **center** of the distribution.



Probability Density Function of the Normal distribution

PDF used to calculate the probability of the **Continuous Random variable** falling within a certain range.

For a **normal distribution** with mean **μ** and standard deviation **σ**, the PDF is given by

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where:

- x is the value for which we want to find the probability density.
- μ is the mean (average) of the distribution.
- σ is the standard deviation, which measures the spread or "width" of the distribution.
- $\exp$ represents the exponential function, which can also be written as $e^{-\text{term}}$, where e is Euler's number (approximately 2.718).

Example - Gaussian Naïve Bayes Classification

Height	Weight	Foot Size	Gender
6.0	180	12	Male
5.92	190	11	Male
5.58	170	12	Male
5.92	165	10	Male
5.0	100	6	Female
5.5	150	8	Female
5.42	130	7	Female
5.75	150	9	Female

.....

Calculate **Mean** and **Standard Deviation** for **Males and Females**

Height	Weight	Foot Size	Gender
6.0	180	12	Male
5.92	190	11	Male
5.58	170	12	Male
5.92	165	10	Male
5.0	100	6	Female
5.5	150	8	Female
5.42	130	7	Female
5.75	150	9	Female

- **Mean** (μ) for a feature:

$$\mu = \frac{\sum_{i=1}^{N} x_i}{N}$$

where x_i is each individual value, and N is the total number of values.

- **Standard Deviation** (σ) for a feature:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}}$$

Step 1: Calculate Mean and Standard Deviation for Males

Height (Male)

1. Mean:

$$\mu_{height,Male} = \frac{6.0 + 5.92 + 5.58 + 5.92}{4}$$

$$= \frac{23.42}{4} = \boxed{5.855}$$

2. Standard Deviation

$$\sigma_{height,Male} = \sqrt{\frac{(6.0 - 5.855)^2 + (5.92 - 5.855)^2 + (5.58 - 5.855)^2 + (5.92 - 5.855)^2}{4}}$$

$$= \sqrt{\frac{(0.021025) + (0.004225) + (0.075625) + (0.004225)}{4}}$$

$$= \sqrt{\frac{0.1051}{4}} = \boxed{0.162}$$

Weight (Male)

1. **Mean:**

$$\mu_{weight,\ Male} = \frac{180+190+170+165}{4}$$

$$= \frac{705}{4} = \boxed{176.25}$$

2. **Standard Deviation**

$$\sigma_{Weight,\ Male} = \sqrt{\frac{(180-176.25)^2 + (190-176.25)^2 + (170-176.25)^2 + (165-176.25)^2}{4}}$$

$$\sigma_{weight,\ male} = \sqrt{\frac{14.0625 + 189.0625 + 39.0625 + 126.5625}{4}}$$

$$= \sqrt{\frac{368.75}{4}}$$

$$= \sqrt{92.1875}$$

$$= \boxed{9.6}$$

Foot size (Male)

1. Mean

$$\mu_{\text{footsize, male}} = \frac{12+11+12+10}{4} = \frac{45}{4}$$

$$= 11.25$$

2. Standard Deviation

$$\sigma_{\text{footsize, male}} = \sqrt{\frac{(12-11.25)^2 + (11-11.25)^2 + (12-11.25)^2 + (10-11.25)^2}{4}}$$

$$= \sqrt{\frac{0.5625 + 0.0625 + 0.5625 + 1.5625}{4}}$$

$$= \sqrt{\frac{2.75}{4}} = \boxed{0.83}$$

Step 2: Calculate Mean and Standard deviation for females

Height (female)

1. Mean

$$\mu_{height,\ female} = \frac{5.0 + 5.5 + 5.42 + 5.75}{4}$$

$$= \frac{21.67}{4} = \boxed{5.4175}$$

2. Standard Deviation:

$$\sigma_{height,\ female} = \sqrt{\frac{(5.0 - 5.4175)^2 + (5.5 - 5.4175)^2 + (5.42 - 5.4175)^2 + (5.75 - 5.4175)^2}{4}}$$

$$= \boxed{0.278}$$

Weight (female)

1. Mean

$$\mu_{\text{weight, female}} = \frac{100 + 150 + 130 + 150}{4}$$

$$= \frac{530}{4} = \boxed{132.5}$$

2. Standard deviation

$$\sigma_{\text{weight, female}} = \sqrt{\frac{(100-132.5)^2 + (150-132.5)^2 + (130-132.5)^2 + (150-132.5)^2}{4}}$$

$$= \boxed{21.6}$$

Foot size (female)

1. Mean

$$\mu_{\text{foot size, female}} = \frac{6+8+7+9}{4}$$

$$= \boxed{7.5}$$

2. Standard deviation

$$\sigma_{\text{foot size, female}} = \sqrt{\frac{(6-7.5)^2 + (8-7.5)^2 + (7-7.5)^2 + (9-7.5)^2}{4}}$$

$$= \boxed{1.29}$$

Male

Height		Weight		Foot size	
μ	σ	μ	σ	μ	σ
5.855	0.162	176.25	9.6	11.25	0.83

Female

Height		Weight		Foot size	
μ	σ	μ	σ	μ	σ
5.4175	0.278	132.5	21.6	7.5	1.29

Step-3 Define Bayes' Theorem

$$\mathbf{P(A|B)} = \frac{P(B|A) * P(A)}{P(B)}$$

$$\boxed{P(\text{Gender}|\text{Features}) = \frac{P(\text{Features}|\text{Gender}) \cdot P(\text{Gender})}{P(\text{Features})}}$$

Here:

- $P(\text{Gender}|\text{Features})$ is the **posterior probability** (probability of gender given features).
- $P(\text{Features}|\text{Gender})$ is the **likelihood** (probability of features given gender).
- $P(\text{Gender})$ is the **prior probability** of each gender.
- $P(\text{Features})$ is the **evidence** (total probability of features).

Step-4 Calculate the Prior Probability

Assume we have 4 males and 4 females in our sample dataset, so:

$$P(\text{Male}) = \frac{4}{8} = 0.5, \quad P(\text{Female}) = \frac{4}{8} = 0.5$$

Step-5 Calculate the Likelihoods Using **Gaussian PDF**

The probability density function for a Gaussian distribution is:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where:

- x is the feature value (e.g., height, weight, foot size),
- μ is the mean of the feature for the gender,
- σ is the standard deviation of the feature for the gender.

Step-6 Calculate the Probability for Each Feature Given Each Gender

- Given a sample data, calculate the **likelihood** for each feature given each gender.

New Data

Height	Weight	Foot size	Gender
6.0	130	8	?

Gaussian Naive bayes Model should Predict the Gender for New data

Likelihood for each feature given each Gender

<u>For Male:</u>

1. <u>Height</u>:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$P(\text{height} = 6.0 \mid \text{Male}) = \frac{1}{\sqrt{2\pi(0.162)^2}} \exp\left(-\frac{(6.0-5.855)^2}{2(0.162)^2}\right)$$

$$P(\text{height} = 6.0 \mid \text{Male}) = \frac{1}{0.4064} \times 0.6703$$

$$\boxed{= 1.65}$$

2. <u>Weight</u>

$$P(\text{weight} = 130 \mid \text{Male}) = \frac{1}{\sqrt{2\pi(9.6)^2}} \exp\left(-\frac{(130-176.25)^2}{2\times(9.6)^2}\right)$$

$$= \frac{1}{24.07} \times 6.6\times10^{-5} = \boxed{0.000276}$$

3 <u>Foot size</u>

$$P(\text{foot size} = 8 \mid \text{Male}) = \frac{1}{\sqrt{2\pi(0.83)^2}} \exp\left(-\frac{(8-11.25)^2}{2\times(0.83)^2}\right)$$

$$P(\text{foot size} = 8 \mid \text{Male}) = \boxed{0.000251}$$

For female

1. **Height**

$$P(\text{height}=6.0|\text{female}) = \frac{1}{\sqrt{2\pi(0.278)^2}} \exp\left(-\frac{(6.0-5.417)^2}{2\times(0.278)^2}\right)$$

$$P(\text{height}=6.0|\text{female}) = \boxed{0.0041}$$

2. **Weight**

$$P(\text{weight}=130|\text{female}) = \frac{1}{\sqrt{2\pi(21.6)^2}} \exp\left(-\frac{(130-132.5)^2}{2\times(21.6)^2}\right)$$

$$P(\text{weight}=130|\text{female}) = \boxed{0.0184}$$

3. <u>Foot size</u>

$$P(\text{Foot size} = 8 \mid \text{Female}) = \frac{1}{\sqrt{2\pi(1.29)^2}} \exp\left(-\frac{(8-7.5)^2}{2\times(1.29)^2}\right)$$

$$P(\text{foot size} = 8 \mid \text{Female}) = \boxed{0.303}$$

Step-7 Combine the Likelihoods

Multiply the likelihoods for each feature to get the combined likelihood for each gender:

$$P(\text{Features}|\text{Male}) = P(\text{height}|\text{Male}) \times P(\text{weight}|\text{Male}) \times P(\text{foot size}|\text{Male})$$

$$P(\text{Features}|\text{Female}) = P(\text{height}|\text{Female}) \times P(\text{weight}|\text{Female}) \times P(\text{foot size}|\text{Female})$$

Step-7 Compute posterior probability using Bayes theorem

Finally, use Bayes' Theorem to get the posterior probabilities:

$$P(\text{Male}|\text{Features}) = \frac{P(\text{Features}|\text{Male}) \cdot P(\text{Male})}{P(\text{Features})}$$

$$P(\text{Female}|\text{Features}) = \frac{P(\text{Features}|\text{Female}) \cdot P(\text{Female})}{P(\text{Features})}$$

Since $P(\text{Features})$ is the same for both classes, we can ignore it for comparison. Whichever posterior probability is higher determines the predicted gender.

Compute Posterior Probability

1. for male

$$P(\text{Male}|\text{Features}) = P(\text{height}|\text{Male}) \times P(\text{weight}|\text{Male}) \times P(\text{foot size}|\text{Male}) \times P(\text{Male})$$

$$= 1.65 \times 0.000276 \times 0.000251 \times 0.5$$

$$= \boxed{5.72 \times 10^{-8}} \Rightarrow 0.000000572$$

2. for female

$$P(\text{female}|\text{Features}) = P(\text{height}|\text{female}) \times P(\text{weight}|\text{female}) \times P(\text{foot size}|\text{female}) \times P(\text{female})$$

$$= 0.0041 \times 0.0184 \times 0.303 \times 0.5$$

$$= \boxed{1.14 \times 10^{-5}} \Rightarrow 0.0000114$$

Step-8 Determine Classification

Since $P(\text{Female}|\text{Features}) > P(\text{Male}|\text{Features})$, the classifier would predict the gender as **Female** for this sample.

Gaussian.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

classify gender based on features like height, weight, and foot size

[1]
```python
import pandas as pd

# Sample dataset
data = {
    'height': [6.0, 5.92, 5.58, 5.92, 5.0, 5.5, 5.42, 5.75],
    'weight': [180, 190, 170, 165, 100, 150, 130, 150],
    'foot_size': [12, 11, 12, 10, 6, 8, 7, 9],
    'gender': ['male', 'male', 'male', 'male', 'female', 'female', 'female', 'female']
}
df = pd.DataFrame(data)
```

```python
#Display the data frame
df
```

	height	weight	foot_size	gender
0	6.00	180	12	male
1	5.92	190	11	male
2	5.58	170	12	male
3	5.92	165	10	male
4	5.00	100	6	female
5	5.50	150	8	female

[4]
```python
#Encode Gender Labels
from sklearn.preprocessing import LabelEncoder
#convert the gender labels into numeric format, where 0 represents female and 1 represents male.
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])
```

```python
df
```

	height	weight	foot_size	gender
0	6.00	180	12	1
1	5.92	190	11	1
2	5.58	170	12	1
3	5.92	165	10	1
4	5.00	100	6	0
5	5.50	150	8	0
6	5.42	130	7	0
7	5.75	150	9	0

[6]

```python
#Split Features and Labels
#Separate the features (height, weight, foot_size) and the label (gender)
X = df[['height', 'weight', 'foot_size']]
y = df['gender']
```

```python
#Train the Gaussian Naive Bayes Classifier
#Using GaussianNB from scikit-learn, train the model on the data.

from sklearn.naive_bayes import GaussianNB

# Initialize the Gaussian Naive Bayes classifier
clf = GaussianNB()
clf.fit(X, y)
```

```
GaussianNB
GaussianNB()
```

```python
#Make Predictions
#After training, you can use the model to predict the gender based on new input data.
# New data for prediction
new_data = [[6.0, 130, 8]]  # height=6.0, weight=130, foot_size=8

# Predict the gender
predicted_gender = clf.predict(new_data)
predicted_gender_label =le.inverse_transform(predicted_gender)

print("Predicted gender:", predicted_gender_label)
```

```
Predicted gender: ['female']
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:493: UserWarning: X does not have valid feature names, but GaussianNB was fitted with feature names
  warnings.warn(
```